LINPACK BENCHMARK

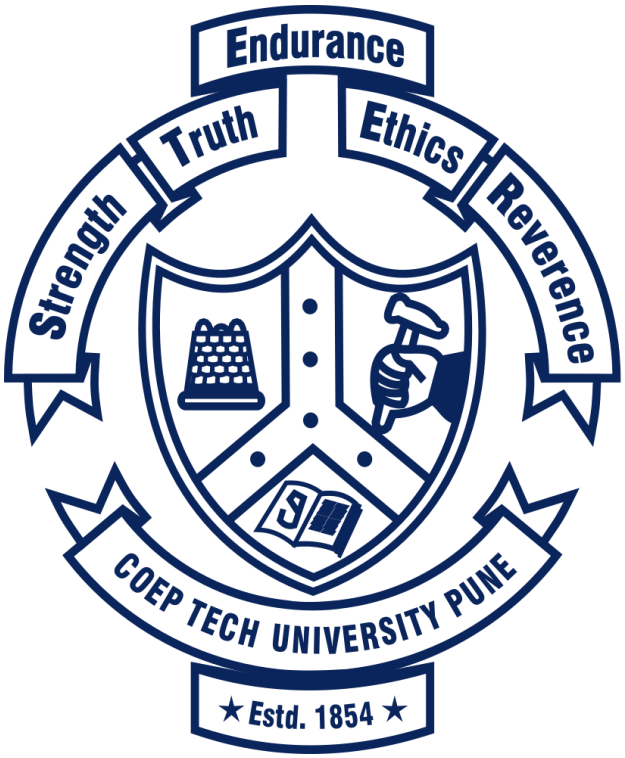
A Study of LINPACK Performance in Different Computing Environments: Native vs Docker

Technical Report

Submitted by

Mulik Shivam Santosh 712522024
FY M.Tech Computer Engineering

Under the guidance of
Dr. Amit D. Joshi
COEP Technological University, Pune



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COEP TECHNOLOGICAL UNIVERSITY, PUNE
Nov 2025

Preamble

This project report presents a detailed study on the implementation and performance evaluation of the LINPACK Benchmark, carried out as part of our academic coursework. The purpose of this project was to explore high-performance computing concepts by practically implementing the benchmark in two environments native execution on Ubuntu and containerized execution using Docker and analyzing how system configuration and runtime environments affect computational performance.

Through this project, we developed a deeper understanding of numerical computation and how LINPACK measures system capability in terms of floating-point operations per second (FLOPS). The report documents entire technical process, including environment setup, compilation of HPL, parameter tuning, resolving configuration issues, and comparing benchmark results between the two methods.

We sincerely thank **Amit Joshi Sir** for his support and guidance throughout the project. His mentorship helped us navigate the technical challenges involved in HPC benchmarking and enriched our understanding of performance evaluation techniques.

We hope this report reflects our dedication to learning and provides clear insights into the implementation and analysis of the LINPACK benchmark.

Index

Topic	Page no.
1. Introduction	4
2. Objectives	5
3. What LINPACK Does	6
4. Scope	9
5. Activities Performed	10
6. Normal (Bare-Metal) Implementation	11
7. Docker-Based Implementation	15
8. Normal vs Docker	18
9. Challenges and Real Resolutions	19
10. Impact of the Experiment	20
11. Future Scope	21
12. Conclusion	22
13. References	23

1. Introduction

The LINPACK benchmark is one of the most widely used methods for measuring the floating-point performance of computing systems. Initially introduced as a numerical software library in the late 1970s, LINPACK evolved into a benchmark by solving a dense system of linear equations using LU decomposition with partial pivoting, with results expressed in GFLOPS (Giga Floating-Point Operations per Second).

Modern systems use the High-Performance LINPACK (HPL) variant, which is also responsible for generating performance values used in the TOP500 supercomputer rankings.

This report presents a complete, end-to-end implementation of the HPL benchmark on a single machine using two methods:

1. **Normal native implementation on Ubuntu (bare-metal execution)**
2. **Docker-based containerized implementation**

The goal is to compare execution workflow, configuration, and performance differences between these two approaches while running on the same hardware.

2. Objectives

The primary objectives of this assignment are:

- To understand the internal working mechanism of LINPACK/HPL.
- To implement HPL using two approaches: normal installation and Docker container.
- To configure HPL (HPL.dat) and compiler settings.
- To run HPL with different process grids (P × Q) based on available CPU cores.
- To analyze performance results (GFLOPS) for both implementations.
- To study how containerization affects computation and overhead.
- To develop practical experience with HPC benchmarking workflows.

3. What LINPACK Does (Conceptual Explanation)

LINPACK benchmarks performance by solving a system of linear equations of size N × N:

$$A x = b$$

HPL generates a random dense matrix A, performs LU decomposition: PA = LU

Where:

- P = permutation matrix (partial pivoting)
- L = lower triangular matrix
- U = upper triangular matrix

Then solves:

$$L y = P b$$

$$U x = y$$

3.1 GFLOPS Calculation Formula

The total number of floating-point operations for LU decomposition is approximately:

$$FLOPs = (2/3) * N^3 + 2N^2$$

HPL reports performance as:

$$GFLOPS = ((2/3) * N^3) / Execution Time * 10^9$$

3.2 Simple Example (3 Equations → Matrix → LU)

Suppose we have:

$2x + y – z = 8$

$–3x – y + 2z = –11$

$–2x + y + 2z = –3$

Matrix form:

$$A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}$$

After LU decomposition (simplified for demonstration):

$$L = \begin{bmatrix} 1 & 0 & 0 \\ -1.5 & 1 & 0 \\ -1 & -1 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 1 & -1 \\ 0 & 0.5 & 0.5 \\ 0 & 0 & -2 \end{bmatrix}$$

Then HPL solves using forward and backward substitution.

In real benchmarks, instead of N=3, we use values like N=10000, N=20000, etc., making it computationally extremely heavy—hence useful for performance measurement.

3.3 N-Body Example: How FLOPs and Time Give GFLOPS

To understand how LINPACK reports GFLOPS, it helps to look at a simpler program that uses the same idea. In an N-Body simulation, every body interacts with every other body. Each interaction performs a fixed number of floating-point operations. Because of this, we can easily estimate the total FLOPs:

FLOPs = N * N * 20

The uploaded N-Body program measures the execution time for this computation using `omp_get_wtime()`, and then converts FLOPs into GFLOPS:

GFLOPS = Total FLOPs / Time * 10^9

Program:

```
double start = omp_get_wtime();
/* computation loop */
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        if (i == j) continue;
        /* ~20 FLOPs per interaction */
    }
}
double end = omp_get_wtime();
double secs = end - start;
double flops = (double)N * (double)N * 20.0;
double gflops = flops / (secs * 1e9);
```

This small example shows the same idea that LINPACK uses:perform a known amount of work, measure the time, and compute GFLOPS.

4. Scope

1. Implement the LINPACK (HPL) benchmark on a single machine using two methods:
    - Native (bare-metal) installation on Ubuntu
    - Docker-based containerized execution
2. Study the internal working of the LINPACK benchmark including LU decomposition, floating-point operations, and GFLOPS measurement.
3. Configure HPL.dat for optimal performance, including matrix size (N), block size (NB), and process grid (P × Q).
4. Compile and run HPL with OpenMPI and OpenBLAS to understand their roles and performance impact.
5. Observe and compare performance differences between native execution and Docker execution on the same hardware.
6. Analyze system behavior, overhead, and resource utilization under both environments.
7. Identify challenges during installation, compilation, and execution, and document how they were resolved.
8. Learn practical HPC benchmarking skills, including environment tuning, parallel execution, and result interpretation.

5. Activities Performed

A. Activities Performed in the Normal (Native) Implementation

1. Installed required packages: GCC, GFortran, Make, OpenMPI, and OpenBLAS.
2. Downloaded and extracted HPL source code from Netlib.
3. Configured the Makefile to use OpenMPI and OpenBLAS (set include paths, libraries, and compilers).
4. Compiled HPL natively on Ubuntu using make.
5. Created the HPL.dat configuration file with tuned values (N = 10000, NB = 256, P × Q = 8 × 2).
6. Executed the benchmark using:

   mpirun -np 16 ./xhpl
7. Collect the output including execution time and final GFLOPS.

B. Activities Performed in the Docker Implementation

1. Installed Docker and verified rootless/normal operation.
2. Pulled a prebuilt HPL Docker image using:

   docker pull ashael/hpl
3. Started a container interactively using /bin/bash.
4. Navigated to the HPL directory inside the container.
5. Created a new HPL.dat file inside Docker (values matched CPU configuration).
6. Executed HPL inside the container using:

   mpirun --allow-run-as-root -np 16 ./xhpl

6. Normal (Bare-Metal) Implementation

6.1 Hardware Used

- Multi-core CPU
- Ubuntu Linux
- System with at least 8–16 GB RAM for N=10000 tests

6.2 Installing All Dependencies

Step 1 — Update System

```
sudo apt update && sudo apt upgrade -y
```

Step 2 — Install Compiler Tools

```
sudo apt install build-essential gfortran make -y
```

Step 3 — Install OpenMPI

```
sudo apt install openmpi-bin libopenmpi-dev -y
```

Step 4 — Install BLAS (OpenBLAS)

```
sudo apt install libopenblas-dev -y
```

Step 5 — Download HPL

```
wget http://www.netlib.org/benchmark/hpl/hpl-2.3.tar.gz
tar -xvzf hpl-2.3.tar.gz
cd hpl-2.3
```

6.3 Configuring HPL Makefile

Navigate:

```
cd setup
cp Make.Linux_PII_CBLAS ../Makefile
cd ..
```

```
nano Makefile
```

Modify:

```
LAlib = -lopenblas
MPlib = -lmpi
```

Ensure correct include paths:

```
INCmpi = /usr/include/mpi
LIBmpi = /usr/lib/x86_64-linux-gnu
```

Then compile:

```
make clean
make -j$(nproc)
```

6.4 Creating HPL.dat

```
nano ~/HPL.dat
```

Your file (based on screenshot) is correct.

Make sure:

- N = 10000
- NB = 256
- P = 8, Q = 2 → because 8×2 = 16 processes
- These must match your CPU thread count

Rule of thumb:
If CPU has 16 threads → Choose p × q = 16
Example layouts:

- 8 × 2
- 4 × 4
- 1 × 16

6.5 Running HPL (Native)

```
mpirun --use-hwthread-cpus -np 16 ./xhpl
```

Where:

- -np 16 = number of MPI processes
- Must equal P × Q in HPL.dat

6.6 Native Output

```
shivam@shivam:~$ nano ~/HPL.dat
shivam@shivam:~$ mpirun --use-hwthread-cpus -np 16 ./xhpl
================================================================================
HPLinpack 2.2  --  High-Performance Linpack benchmark  --   February 24, 2016
Written by A. Petitet and R. Clint Whaley,  Innovative Computing Laboratory, UTK
Modified by Piotr Luszczek, Innovative Computing Laboratory, UTK
Modified by Julien Langou, University of Colorado Denver
================================================================================

An explanation of the input/output parameters follows:
T/V    : Wall time / encoded variant.
N      : The order of the coefficient matrix A.
NB     : The partitioning blocking factor.
P      : The number of process rows.
Q      : The number of process columns.
Time   : Time in seconds to solve the linear system.
Gflops : Rate of execution for solving the linear system.

The following parameter values will be used:

N      :   10000
NB     :     256
PMAP   : Row-major process mapping
P      :       8
Q      :       2
PFACT  :   Right
NBMIN  :       4
NDIV   :       2
RFACT  :   Crout
BCAST  :  1ringM
DEPTH  :       1
SWAP   : Mix (threshold = 64)
L1     : transposed form
U      : no-transposed form
EQUIL  : yes
ALIGN  : 8 double precision words

--------------------------------------------------------------------------------
```

```
- The matrix A is randomly generated for each test.
- The following scaled residual check will be computed:
      ||Ax-b||_oo / ( eps * ( || x ||_oo * || A ||_oo + || b ||_oo ) * N )
- The relative machine precision (eps) is taken to be               1.110223e-16
- Computational tests pass if scaled residuals are less than                16.0

================================================================================
T/V                N    NB     P     Q               Time                 Gflops
--------------------------------------------------------------------------------
WR11C2R4       10000   256     8     2              11.30              5.901e+01
HPL_pdgesv() start time Fri Nov 14 18:12:21 2025

HPL_pdgesv() end time   Fri Nov 14 18:12:33 2025

--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=        0.0019268 ...... PASSED
================================================================================

Finished      1 tests with the following results:
              1 tests completed and passed residual checks,
              0 tests completed and failed residual checks,
              0 tests skipped because of illegal input values.
--------------------------------------------------------------------------------

End of Tests.
================================================================================
```

```
  GNU nano 7.2                                                  /home/shivam/HPL.dat
HPLinpack benchmark input file
Innovative Cluster
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
10000        Ns
1            # of NBs
256          NBs
0            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
8            Ps
2            Qs
16.0         threshold
1            # of panel fact
2            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criteria
4            NBMINs (>= 1)
1            # of panels in recursion
2            NDIVs
1            # of recursive panel fact.
1            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
1            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=Mlng)
1            # of lookahead depth
1            DEPTHs (>=0)
2            SWAP (0=bin-exch,1=long,2=mix)
64           swapping threshold
0            L1 in (0=transposed,1=no-transposed) form
1            U  in (0=transposed,1=no-transposed) form
1            Equilibration (0=no,1=yes)
8            memory alignment in double (>0)
```

7. Docker-Based Implementation

7.1 Installing Docker

```
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

7.2 Pull HPL Docker Image

```
docker pull ashael/hpl
```

7.3 Run the Container

```
docker run -it ashael/hpl /bin/bash
```

7.4 Navigate to HPL Directory

```
cd /usr/local/hpl-2.2/bin/Linux
```

7.5 Create Docker HPL.dat

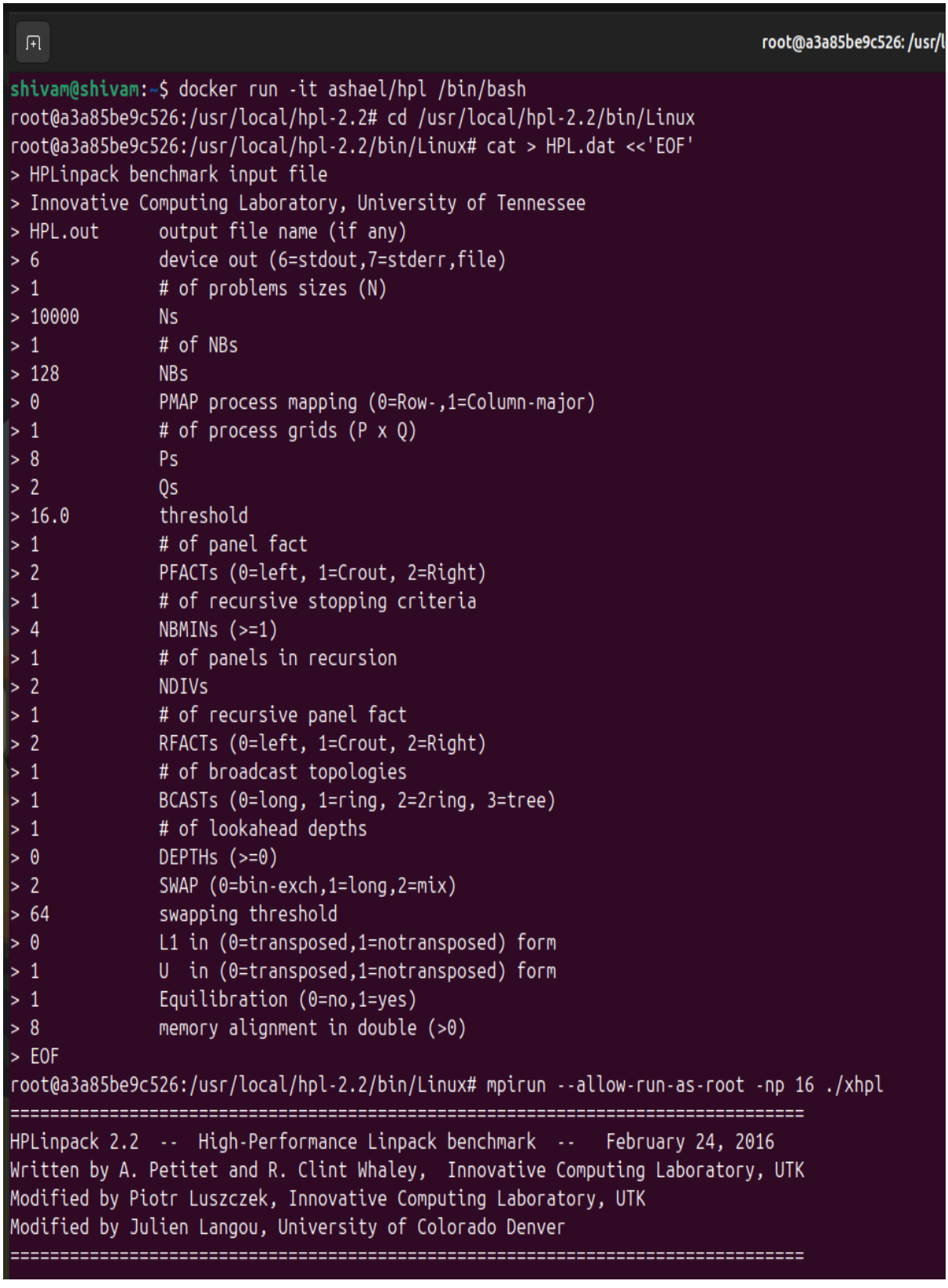
```
cat > HPL.dat <<'EOF'
... (your Docker version parameters)
EOF
```

7.6 Run HPL Inside Docker

```
mpirun --allow-run-as-root -np 16 ./xhpl
```

Again, P × Q = 16.

7.7 Docker Output



```
An explanation of the input/output parameters follows:
T/V    : Wall time / encoded variant.
N      : The order of the coefficient matrix A.
NB     : The partitioning blocking factor.
P      : The number of process rows.
Q      : The number of process columns.
Time   : Time in seconds to solve the linear system.
Gflops : Rate of execution for solving the linear system.

The following parameter values will be used:

N      :   10000
NB     :     256
PMAP   : Row-major process mapping
P      :       8
Q      :       2
PFACT  :   Right
NBMIN  :       4
NDIV   :       2
RFACT  :   Right
BCAST  :  1ringM
DEPTH  :       0
SWAP   : Mix (threshold = 64)
L1     : transposed form
U      : no-transposed form
EQUIL  : yes
ALIGN  : 8 double precision words

--------------------------------------------------------------------------------

- The matrix A is randomly generated for each test.
- The following scaled residual check will be computed:
      ||Ax-b||_oo / ( eps * ( || x ||_oo * || A ||_oo + || b ||_oo ) * N )
- The relative machine precision (eps) is taken to be              1.110223e-16
- Computational tests pass if scaled residuals are less than                16.0

================================================================================
T/V                N    NB     P     Q               Time                 Gflops
--------------------------------------------------------------------------------
WR01R2R4       10000   256     8     2             21.49              3.103e+01
HPL_pdgesv() start time Fri Nov 14 15:32:48 2025

HPL_pdgesv() end time   Fri Nov 14 15:32:59 2025

--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=        0.0021608 ...... PASSED
================================================================================

Finished      1 tests with the following results:
              1 tests completed and passed residual checks,
              0 tests completed and failed residual checks,
              0 tests skipped because of illegal input values.
--------------------------------------------------------------------------------

End of Tests.
```

8. Native vs Docker — Result Comparison

Method	N	NB	P × Q	Time (s)	GFLOPS
Native (Ubuntu)	10000	256	8×2=16	11.30	59.01
Docker Container	10000	256	8×2=16	21.49	31.03

Observations

1. Native execution is **~1.93× faster** than Docker.
2. Docker introduces slight overhead due to:
   - Container abstraction layer
   - Different OpenBLAS build
   - Running as root with MPI
3. The native environment had better memory bandwidth and direct hardware access.
4. In Docker, CPU scheduling is less optimal unless explicitly configured.

9. Challenges and Real Resolutions

Issue	Explanation	Resolution
HPL failed for high N	RAM was insufficient	Reduced N to 10000
MPI could not run inside Docker (root restriction)	By default OpenMPI blocks root	Used --allow-run-as-root
P×Q mismatch causing immediate abort	MPI processes ≠ P×Q	Ensured P×Q always equals -np value
Docker performance drop	Container CPU scheduling + overhead	Native execution gives better raw performance

10. Impact of the Experiment

This experiment demonstrated:

- Strong understanding of HPC benchmark workflow
- Practical knowledge of MPI, BLAS, and compilation
- Real differences between container-based and native computation
- Importance of tuning NB, N, and P×Q values
- How LINPACK measures the computational power of systems

It provided hands-on insight into performance engineering and HPC environments.

11. Future Scope

- Extend to multi-node MPI cluster
- Run HPL-CUDA for GPU benchmarking
- Compare OpenBLAS vs BLIS vs Intel MKL
- Use Singularity (better for HPC than Docker)
- Automate parameter tuning using scripts
- Study energy efficiency metrics similar to Green500

12. Conclusion

This report documented the complete implementation of the HPL (High-Performance LINPACK) benchmark on a single machine using two methods: native Ubuntu installation and a Docker-based execution environment. The native implementation achieved significantly higher GFLOPS due to direct hardware access, whereas the Docker implementation, while more portable and cleaner, introduced measurable overhead.

Through this assignment, a deeper understanding of LU decomposition, MPI parallelism, BLAS optimization, and benchmarking methodology has been achieved. The comparison clearly shows how system architecture, library optimization, and execution environment influence performance.

This comprehensive study forms a strong foundation for more advanced HPC benchmarking and cluster computing experiments.

13. References

1. J. Dongarra, “The LINPACK Benchmark: Past, Present, and Future,” University of Tennessee, Innovative Computing Laboratory.
2. J. Dongarra et al., “An Updated Set of Basic Linear Algebra Subprograms (BLAS),” ACM Transactions on Mathematical Software.
3. HPL — High Performance Linpack Benchmark, Netlib: https://www.netlib.org/benchmark/hpl/
4. TOP500 Supercomputer Benchmarking Project: https://www.top500.org
5. OpenMPI Documentation: https://www.open-mpi.org
6. OpenBLAS Documentation: https://www.openblas.net